

Java User Input (Scanner)

The Scanner class is used to get user input, and it is found in the `java.util` package.

To use the Scanner class, create an object of the class and use any of the available methods found in the Scanner class documentation. In our example, we will use the `nextLine()` method, which is used to read Strings

```
import java.util.Scanner; // Import the Scanner class
```

```
class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner myObj = new Scanner(System.in); // Create a  
Scanner object
```

```
        System.out.println("Enter username");
```

```
        String userName = myObj.nextLine(); // Read user input
```

```
        System.out.println("Username is: " + userName); //
```

```
Output user input
```

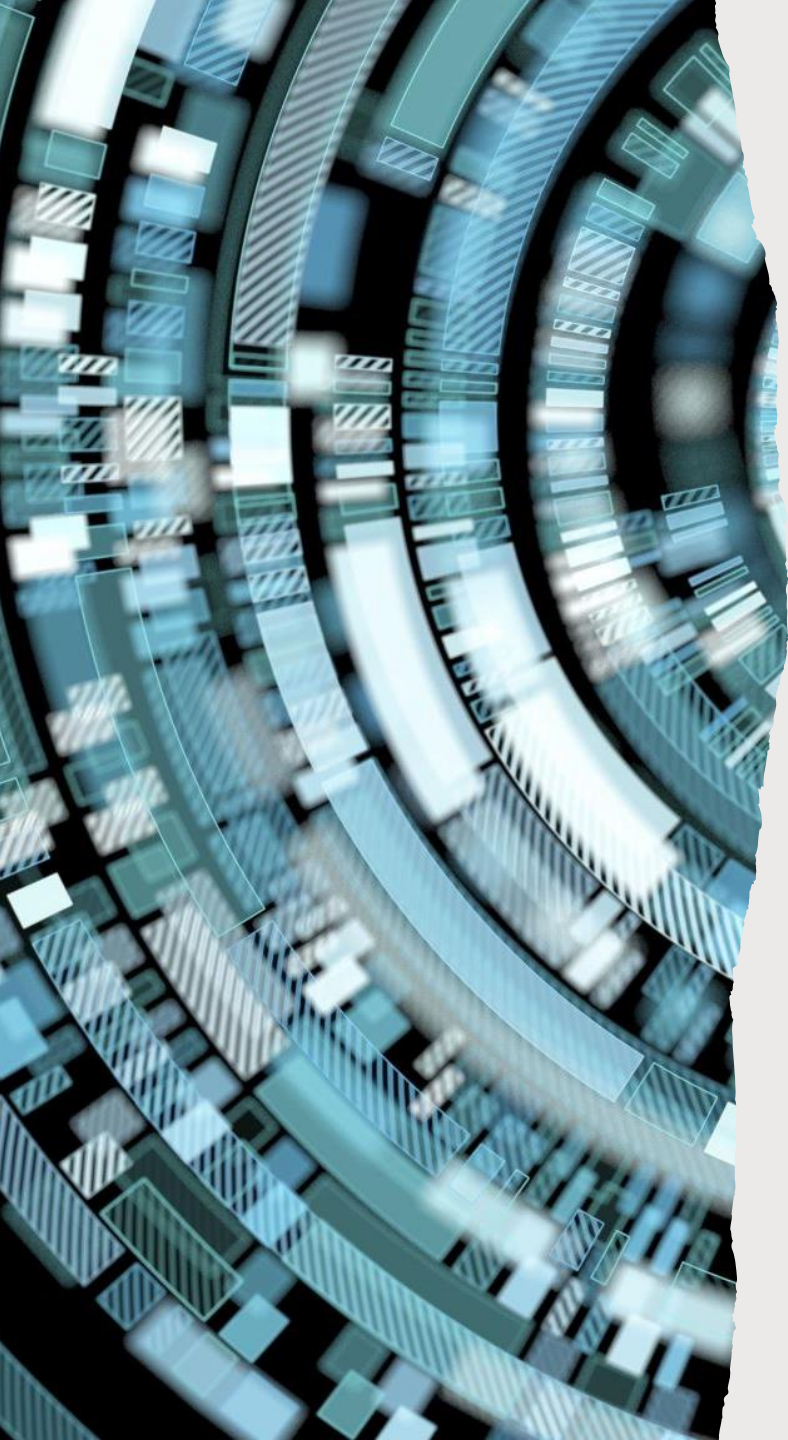
```
    }
```

```
}
```

Input Types

In the example above, we used the `nextLine()` method, which is used to read Strings. To read other types, look at the table below:

Method	Description
<code>nextBoolean()</code>	Reads a <code>boolean</code> value from the user
<code>nextByte()</code>	Reads a <code>byte</code> value from the user
<code>nextDouble()</code>	Reads a <code>double</code> value from the user
<code>nextFloat()</code>	Reads a <code>float</code> value from the user
<code>nextInt()</code>	Reads a <code>int</code> value from the user
<code>nextLine()</code>	Reads a <code>String</code> value from the user
<code>nextLong()</code>	Reads a <code>long</code> value from the user
<code>nextShort()</code>	Reads a <code>short</code> value from the user



- **Polymorphism
in
java**

Polymorphism

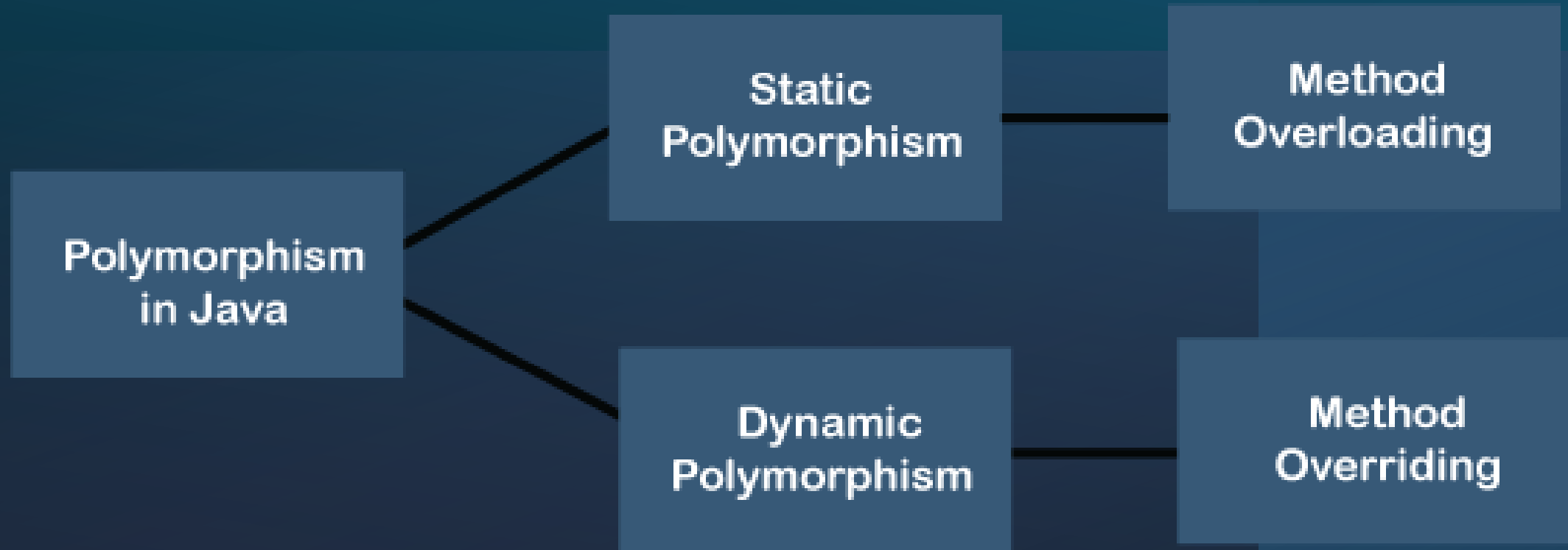
The word polymorphism is a combination of two words i.e. ploy and morphs. The word poly means many and morphs means different forms. In short, a mechanism by which we can perform a single action in different ways.



Polymorphism



- **Static Polymorphism (Compile Time Polymorphism)**
- **Dynamic Polymorphism (Run Time Polymorphism)**



Method Overloading

- Method overloading occurs within a single class and enables a class to have more than one method with the same name, as long as their parameter lists are different.

```
class Cat{  
  
    public void Sound(){  
        System.out.println("meow");  
    }  
  
    //overloading method  
    public void Sound(int num){  
        for(int i=0; i<2;i++){  
            System.out.println("meow");  
        }  
    }  
}
```

OVERLOADING

**Same method
name but
different
parameters**




```
class ExampleClass {  
    void display(int a) {  
        System.out.println("Display with integer: " + a);  
    }  
  
    void display(String b) {  
        System.out.println("Display with string: " + b);  
    }  
}
```

Key Points:

- **Same method name, different parameters (either in type, number, or both).**
- **Implemented within the same class.**
- **Helps in increasing the readability of the program.**
- **It's an example of static polymorphism or compile-time polymorphism.**

Scenario: Online Bookstore System

You are tasked with designing a part of an online bookstore system. The system should allow users to search for books based on different criteria: by title, by author, or by both title and author. To implement this functionality, you decide to use method overloading.

Your job is to create a class named `BookSearch` that includes overloaded methods for searching books. Each method should simulate a search operation by printing out a message indicating which search criteria are being used. For simplicity, assume that the actual search logic and book database are handled elsewhere in the system, and you are only responsible for the method signatures and their immediate output.

```
public class BookSearch {

    // Method to search books by title
    public void search(String title) {
        System.out.println("Searching for books with title: " + title);
        // Simulate search logic...
    }

    // Method to search books by author
    public void searchByAuthor(String author) {
        System.out.println("Searching for books by author: " + author);
        // Simulate search logic...
    }

    // Overloaded method to search books by both title and author
    public void search(String title, String author) {
        System.out.println("Searching for books with title: \"" + title + "\" and author: " + author);
        // Simulate search logic...
    }

    public static void main(String[] args) {
        BookSearch bookSearch = new BookSearch();

        // Test the search by title
        bookSearch.search("The Great Gatsby");

        // Test the search by author
        bookSearch.searchByAuthor("F. Scott Fitzgerald");

        // Test the search by both title and author
        bookSearch.search("The Great Gatsby", "F. Scott Fitzgerald");
    }
}
```

Scenario: Online Store Payment Processor

The online store needs to process payments through various methods. The `PaymentProcessor` class should be able to handle payments by:

Credit Card: Requires the card number, expiration date, and CVV code.

PayPal: Requires the user's PayPal email.

Loyalty Points: Requires the user's account ID and the number of points to use.

```
class PaymentProcessor {  
    // Method to process payment by Credit Card  
    public void processPayment(String cardNumber, String  
expiryDate, int cvv) {  
        if (cardNumber.length() == 16 &&  
expiryDate.matches("\\d{2}/\\d{2}") && (cvv >= 100 && cvv <= 999)) {  
            System.out.println("Processing credit card payment. Card  
Number: " + cardNumber);  
            // Simulated successful payment processing...  
            System.out.println("Payment successful with Credit Card.");  
        } else {  
            System.out.println("Invalid credit card details provided.");  
        }  
    }  
}
```

// Method to process payment by PayPal

```
public void processPayment(String paypalEmail) {  
    if (paypalEmail.contains("@")) {  
        System.out.println("Processing PayPal payment. PayPal Email: " + paypalEmail);  
        // Simulated successful payment processing...  
        System.out.println("Payment successful with PayPal.");  
    } else {  
        System.out.println("Invalid PayPal email provided.");  
    }  
}
```

// Method to process payment using Loyalty Points

```
public void processPayment(String accountId, int points) {  
    if (accountId != null && !accountId.trim().isEmpty() && points >= 1000) { // Assuming 1000 points as the minimum  
for a transaction  
        System.out.println("Processing payment with Loyalty Points. Account ID: " + accountId + ", Points: " + points);  
        // Simulated successful payment processing...  
        System.out.println("Payment successful with Loyalty Points.");  
    } else {  
        System.out.println("Invalid account details or insufficient points.");  
    }  
}
```

```
public class OnlineStore {  
    public static void main(String[] args) {  
        PaymentProcessor paymentProcessor = new PaymentProcessor();  
  
        // Example of processing different types of payments  
        paymentProcessor.processPayment("1234567890123456", "12/23", 123); // Valid  
Credit Card  
        paymentProcessor.processPayment("invalid-cc", "99/99", 0000); // Invalid Credit  
Card  
  
        paymentProcessor.processPayment("user@example.com"); // Valid PayPal  
        paymentProcessor.processPayment("notAnEmail"); // Invalid PayPal  
  
        paymentProcessor.processPayment("user123", 5000); // Valid Loyalty Points  
        paymentProcessor.processPayment("user123", 500); // Invalid Loyalty Points  
(Insufficient)  
    }  
}
```


Method Overriding

- **Method overriding, on the other hand, occurs when a subclass or child class has a method with the same name, return type, and parameters as a method in its superclass or parent class.**
- **It allows a subclass to provide a specific implementation of a method that is already provided by its superclass.**
- **Method overriding is a runtime polymorphism technique, meaning the method that gets executed is determined at runtime based on the object's runtime type.**

Method Overriding

It provides a specific implementation to a method that is already present in the parent class. it is used to achieve run-time polymorphism. Remember that, it is not possible to override the static method. Hence, we cannot override the main() method also because it is a static method.

Rules for Method Overriding

- ❖ The name of the method must be the same as the name of the parent class method.
- ❖ The number of parameters and the types of parameters must be the same as in the parent class.
- ❖ There must exist an IS-A relationship (inheritance).
- ❖ We call an overridden method through a reference of the parent class. The type of object decides which method is to be executed and it is decided by the JVM at runtime.

Key Points:

- ☐ Same method name, same parameters as in the superclass.
- ☐ Occurs between superclass and subclass.
- ☐ Allows a subclass to provide a specific implementation of a method.
- ☐ It's an example of dynamic polymorphism or runtime polymorphism.

Dynamic Polymorphism

Dynamic polymorphism is a process or mechanism in which a call to an overridden method is to resolve at runtime rather than compile-time. It is also known as runtime polymorphism or dynamic method dispatch. We can achieve dynamic polymorphism by using the method overriding.

In this process, an overridden method is called through a reference variable of a superclass. The determination of the method to be called is based on the object being referred to by the reference variable.

Properties of Dynamic Polymorphism:

It decides which method is to execute at runtime.

It can be achieved through dynamic binding.

It happens between different classes.

It is required where a subclass object is assigned to a super-class object for dynamic polymorphism.

Inheritance involved in dynamic polymorphism.

```
class SuperClass {  
    void display() {  
        System.out.println("Display in SuperClass");  
    }  
}
```

```
class SubClass extends SuperClass {  
    @Override  
    void display() {  
        System.out.println("Display in SubClass");  
    }  
}
```

```
//parent class  
class Sample {  
    public void display() {  
        System.out.println("Overridden Method");    } }  
  
public class Demo extends Sample {  
    public void display() {  
        @Override  
        System.out.println("Overriding Method");    }  
    public static void main(String args[]) {  
        //assigning a child class object to parent class reference  
        Sample obj = new Demo();  
        //invoking display() method  
        obj.display();    } }
```

Example of Method Overriding

```
public class DynamicPolymorphismExample {
public static void main(String args[]) {
//assigning a child class object to a parent class reference
Fruits fruits = new Mango();
//invoking the method
fruits.color();
} }

//parent class
class Fruits {
public void color() {
System.out.println("Parent class method is invoked."); } }

//derived or child class that extends the parent class
class Mango extends Fruits {
//overrides the color() method of the parent class
@Override
public void color() {
System.out.println("The child class method is invoked."); } }
```

Scenario: Animal Sound Simulator

Imagine you're building a program to simulate the sounds made by different animals. You want to create a system where each type of animal can produce its own unique sound. This scenario will help you understand how method overriding and dynamic polymorphism work in Java.


```
// Base class representing an Animal
class Animal {
    // Method to make a sound
    public void makeSound() {
        System.out.println("Animal makes a sound.");
    }
}
```

```
// Derived class representing a Dog
class Dog extends Animal {
    // Override method to make a bark sound
    @Override
    public void makeSound() {
        System.out.println("Dog barks: Woof! Woof!");
    }
}
```

```
// Derived class representing a Cat
class Cat extends Animal {
    // Override method to make a meow sound
    @Override
    public void makeSound() {
        System.out.println("Cat meows: Meow! Meow!");
    }
}
```

```
// Main class
public class AnimalSoundSimulator {
    public static void main(String[] args) {
        // Create instances of Dog and Cat
        Dog dog = new Dog();
        Cat cat = new Cat();

        // Call makeSound method on Dog and Cat objects
        dog.makeSound(); // Output will be: Dog barks: Woof!
        Woof!
        cat.makeSound(); // Output will be: Cat meows: Meow!
        Meow!
    }
}
```

Scenario: Shape Area Calculator

Imagine you are developing a program to calculate the area of different geometric shapes. You want to create a system where each type of shape (like Circle, Rectangle, and Triangle) can calculate its area based on its specific properties. This scenario will help you understand how method overriding and dynamic polymorphism can be applied in a practical context.

```
// Base class representing a Shape
class Shape {
    // Method to calculate area
    public double calculateArea() {
        return 0; // Default implementation for undefined shapes
    }
}
```

```
// Derived class representing a Circle
class Circle extends Shape {
    private double radius;

    // Constructor
    public Circle(double radius) {
        this.radius = radius;
    }

    // Override method to calculate area of circle
    @Override
    public double calculateArea() {
        return Math.PI * radius * radius;
    }
}
```

```
// Derived class representing a Rectangle
class Rectangle extends Shape {
    private double length;
    private double width;

    // Constructor
    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    // Override method to calculate area of rectangle
    @Override
    public double calculateArea() {
        return length * width;
    }
}
```

```
// Derived class representing a Triangle
class Triangle extends Shape {
    private double base;
    private double height;
```

```
// Constructor
```

```
public Triangle(double base, double height) {  
    this.base = base;  
    this.height = height;  
}
```

```
// Override method to calculate area of triangle
```

```
@Override
```

```
public double calculateArea() {  
    return 0.5 * base * height;  
}  
}
```

```
// Main class
```

```
public class ShapeAreaCalculator {  
    public static void main(String[] args) {  
        // Create instances of Circle, Rectangle, and Triangle  
        Circle circle = new Circle(5);  
        Rectangle rectangle = new Rectangle(4, 6);  
        Triangle triangle = new Triangle(3, 4);  
  
        // Calculate and print areas of shapes  
        System.out.println("Area of Circle: " +  
circle.calculateArea());  
        System.out.println("Area of Rectangle: " +  
rectangle.calculateArea());  
        System.out.println("Area of Triangle: " +  
triangle.calculateArea());  
    }  
}
```

ABSTRACTION

ABSTRACTION

Abstraction in Java is a concept that helps in hiding the complex reality while exposing only the necessary features of an object. In Java, abstraction is achieved using abstract classes and interfaces.



Fig: Realtime Example of Abstraction in Java

Abstract Classes

An abstract class in Java is a class that cannot be instantiated on its own and is meant to be subclassed. It may contain abstract methods as well as concrete (fully defined) methods. Abstract classes are used to represent generic concepts or base behaviors that subclasses should implement or expand upon.

Characteristics of Abstract Classes:

- ❖ **Cannot be instantiated:** You cannot create objects of an abstract class directly.
- ❖ **Can have constructors:** Abstract classes can have constructors, and the constructor is called when a subclass is instantiated.
- ❖ **Can have instance variables:** Abstract classes can declare fields that are not static and final, and define methods that can access and modify the state of the object to which they belong.
- ❖ **Can have defined methods:** Abstract classes can have fully implemented methods with actual code.
- ❖ **Can have abstract methods:** Methods without a body, just declared with the keyword `abstract` and a semicolon at the end.

Abstract Methods

Abstract methods are methods **declared in an abstract class that do not have an implementation**. Subclasses that extend an abstract class must provide implementations for its abstract methods unless the subclass is also abstract.

Characteristics of Abstract Methods:

- No body:** Abstract methods do not have a body; they're just a signature followed by a semicolon.
- Implementation by subclasses:** Any concrete subclass must implement all inherited abstract methods, or it too must be declared abstract.
- Enforces a contract:** Declaring an abstract method in a class imposes a contract on the subclasses, obliging them to support particular behaviors.

The Purpose of Abstraction:

- ❖ **To control the complexity of large software systems by hiding the implementation details and exposing only the necessary parts of an object.**
- ❖ **To provide a common template for subclasses that can be adapted and implemented as per the subclass's specific requirements.**
- ❖ **To enable code reuse by defining non-specific method signatures in the parent class that different child classes can share and implement in different ways.**

```
abstract class Shape {
    abstract double area(); }
class Triangle extends Shape {
    double base, height;
    Triangle(double base, double height) {
        this.base = base;
        this.height = height; }
    @Override
    double area() {
        return 0.5 * base * height; } }
class Rectangle extends Shape {
    double length, width;
    Rectangle(double length, double width) {
        this.length = length;
        this.width = width; }
    @Override
    double area() {
        return length * width; }}
public class GeometryTest {
    public static void main(String[] args) {
        Shape triangle = new Triangle(3, 4);
        Shape rectangle = new Rectangle(5, 6);
        System.out.println("Triangle area: " + triangle.area());
        System.out.println("Rectangle area: " + rectangle.area()); }}
```

```
// Declares an abstract class named Shape. An abstract class cannot be instantiated.
abstract class Shape {
    // Declares an abstract method named area. It has no body and must be overridden in subclasses.
    abstract double area();
}

// Declares a concrete class named Triangle that extends the abstract Shape class.
class Triangle extends Shape {
    // Instance variables to store the dimensions of the triangle.
    double base, height;
    // Constructor for the Triangle class. It initializes the base and height of the triangle.
    Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }
    // Overrides the abstract area method from the Shape class. Calculates and returns the area of the triangle.
    @Override
    double area() {
        return 0.5 * base * height;
    }
}

// Declares a concrete class named Rectangle that extends the abstract Shape class.
class Rectangle extends Shape {
    // Instance variables to store the dimensions of the rectangle.
    double length, width;

    // Constructor for the Rectangle class. It initializes the length and width of the rectangle.
    Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }
    // Overrides the abstract area method from the Shape class. Calculates and returns the area of the rectangle.
    @Override
    double area() {
        return length * width;
    }
}

// Contains the main method, which is the entry point of the program.
public class GeometryTest {
    public static void main(String[] args) {
        // Creates a Triangle object with base 3 and height 4, referenced by a Shape variable.
        Shape triangle = new Triangle(3, 4);
        // Creates a Rectangle object with length 5 and width 6, referenced by a Shape variable.
        Shape rectangle = new Rectangle(5, 6);
        // Calls the area method on the triangle object and prints the result.
        System.out.println("Triangle area: " + triangle.area());
        // Calls the area method on the rectangle object and prints the result.
        System.out.println("Rectangle area: " + rectangle.area());
    }
}
```

```
// Define an abstract class for a generic payment processor.
abstract class PaymentProcessor {
    // Abstract method that needs to be implemented by subclasses to process payments.
    abstract void processPayment(double amount);
}

// Subclass of PaymentProcessor for processing credit card payments.
class CreditCardProcessor extends PaymentProcessor {
    // Implement the processPayment method for credit card processing.
    @Override
    void processPayment(double amount) {
        System.out.println("Processing credit card payment of $" + amount);
    }
}

// Subclass of PaymentProcessor for processing payments through PayPal.
class PayPalProcessor extends PaymentProcessor {
    // Implement the processPayment method for PayPal processing.
    @Override
    void processPayment(double amount) {
        System.out.println("Processing PayPal payment of $" + amount);
    }
}

// Main class to test the payment processing subclasses.
public class PaymentTest {
    public static void main(String[] args) {
        // Create an instance of CreditCardProcessor.
        PaymentProcessor creditCard = new CreditCardProcessor();
        // Create an instance of PayPalProcessor.
        PaymentProcessor paypal = new PayPalProcessor();

        // Process a payment using the credit card processor.
        creditCard.processPayment(100.0);
        // Process a payment using the PayPal processor.
        paypal.processPayment(200.0);
    }
}
```

Basic Inheritance

Write a Java program where class Vehicle is the superclass with a method displayMaxSpeed() that prints out a default speed. Class Car extends Vehicle and overrides the displayMaxSpeed() method to print out a higher speed.

Method Overriding

Write a Java program with a superclass Plant that has a method needsWater() which returns a boolean value. A subclass Cactus should override this method to return false, signifying that cacti require less water.

Multi-level Inheritance

Create a Java program with three levels of inheritance. At the top, have a class Instrument with a method play(). Below it, have a class StringInstrument that extends Instrument. Finally, have a class Guitar that extends StringInstrument and overrides the play() method to print "Guitar is playing chords".

Super Keyword

Write a Java program that demonstrates the use of the super keyword to access a superclass's constructor and methods. Create a superclass Food with a property taste and a method printTaste(). Then, create a subclass Fruit that uses super to set the taste and to invoke the printTaste() method of Food.